№ 12(53) 2025
НАУКА і ТЕХНІКА СЬОГОДНІ
серії: право, економіка, педагогіка, техніка, фізико-математичні науки

UDC 004.8

https://doi.org/10.52058/2786-6025-2025-12(53)-109-118

Yuskovych-Zhukovska Valentyna Ivanivna, PhD of technical science, docent, Academician Stepan Demianchuk International University of Economics and Humanities (Ukraine), https://orcid.org/0000-0002-4236-1467

Bogut Oleg Mykhailovych, senior lecturer, Academician Stepan Demianchuk International University of Economics and Humanities (Ukraine), https://orcid.org/0000-0002-0426-6303

METHODS OF PREVENTING INFORMATION LEAKS CAUSED BY THE USAGE OF ARTIFICIAL INTELLIGENCE AGENT SYSTEMS

Abstract. The article investigates the growing problem of information leakage in agent-based artificial intelligence systems, whose adoption in business processes, scientific workflows, and corporate IT infrastructures is rapidly increasing. The relevance of the research arises from the unique architecture of agentic systems, where autonomous modules cooperate, exchange intermediate data, and possess tool-level privileges, creating leakage vectors that fundamentally differ from those of traditional software or standalone large language models. As organizations increasingly rely on multi-agent automation for handling documents, browsing, computation, and decision support, ensuring controlled information flows becomes a critical security challenge.

The study is based on the hypothesis that information leakage in agent-based AI systems can be significantly minimized through an integrated defense-in-depth architecture that combines least-privilege access control, trust-segmented communication, input/output sanitization, policy-as-code governance, and continuous monitoring of tool interactions. According to this hypothesis, leakage reduction is not achieved by a single mechanism but by the interaction of several orthogonal controls that collectively constrain both intentional and accidental disclosure.

To verify the hypothesis, the article develops a formal threat model and a taxonomy of leakage channels, covering prompt injections, insecure output handling, instrumental exfiltration through APIs or files, inter-agent collusion, memory/history exposure, and unintended training memorization. An experimental methodology is constructed using representative multi-agent tasks and standardized attack suites. Leakage incidents are measured through a set of quantitative metrics, including frequency, disclosure volume, and mean time to detection, with evaluation supported by statistical tests, ablation studies, and controlled scenario repetition.

The results confirm that the proposed multilayered defense architecture substantially reduces leakage risks: attack success rates, disclosure volumes, and detection times improved consistently across all tested scenarios. The findings validate the efficiency of combined privilege control, egress restrictions, intent verification, and context isolation as core components of robust leakage prevention in contemporary AI-driven agent systems.

Keywords: agent-based artificial intelligence; multi-agent systems; information leakage; data loss prevention (DLP) systems; access control; secure communication.

Юскович-Жуковська Валентина Іванівна, кандидат технічних наук, доцент, Міжнародний економіко-гуманітарний університет імені академіка Степана Дем'янчука (Україна), https://orcid.org/0000-0002-4236-1467

Богут Олег Михайлович, старший викладач, Міжнародний економіко-гуманітарний університет імені академіка Степана Дем'янчука (Україна), https://orcid.org/0000-0002-0426-6303

МЕТОДИ ЗАПОБІГАННЯ ВИТІКАМ ІНФОРМАЦІЇ, СПРИЧИНЕНИМ ВИКОРИСТАННЯМ АГЕНТНИХ СИСТЕМ ШТУЧНОГО ІНТЕЛЕКТУ

Анотація. У статті досліджується зростаюча проблема витоку інформації в агентних системах штучного інтелекту, застосування яких у бізнес-процесах, наукових робочих циклах та корпоративних IT-інфраструктурах стрімко зростає. Актуальність дослідження зумовлена унікальною архітектурою агентних систем, у яких автономні модулі взаємодіють, обмінюються проміжними даними та володіють інструментальними привілеями, формуючи вектори витоку, що принципово відрізняються від традиційного програмного забезпечення або окремих моделей штучного інтелекту. Із посиленням залежності організацій від багатоагентної автоматизації для обробки документів, навігації, обчислень і підтримки рішень забезпечення контрольованості інформаційних потоків стає критично важливим викликом безпеки.

Дослідження ґрунтується на гіпотезі, що витоки інформації в агентних AI-системах можуть бути суттєво мінімізовані завдяки інтегрованій багаторівневій архітектурі захисту, яка поєднує принцип найменших привілеїв, сегментацію довіри, санітизацію вхідних і вихідних даних, policy-as-code та безперервний моніторинг взаємодії інструментів. Відповідно до цієї гіпотези, зменшення витоків досягається не окремим механізмом, а взаємодією кількох ортогональних засобів контролю, що одночасно обмежують як ненавмисні, так і навмисні розголошення.

Для перевірки гіпотези розроблено формалізовану модель загроз і таксономію каналів витоку, що охоплюють ін'єкції запитів, небезпечну обробку вихідних даних, інструментальну ексфільтрацію через API чи файли, міжагентну змову, витоки з пам'яті та історії, а також ненавмисну меморизацію під час навчання. Побудовано експериментальну методологію на основі типових багатоагентних сценаріїв і стандартизованих наборів атак. Інциденти витоків оцінювалися за кількісними метриками — частотою, обсягом розкритих даних і середнім часом виявлення — з використанням статистичних тестів, абляційних досліджень та повторюваних сценарних перевірок.

Результати підтверджують, що запропонована багаторівнева архітектура захисту суттєво зменшує ризики витоків: частота успішних атак, обсяги розкритих даних і час до виявлення покращилися в усіх протестованих сценаріях. Отримані висновки підтверджують ефективність поєднання контролю привілеїв, обмеження вихідного трафіку, перевірки намірів та ізоляції контекстів як ключових компонентів надійного запобігання витокам у сучасних агентних AI-системах.

Ключові слова: агентний штучний інтелект; багатоагентні системи; витік інформації; системи запобігання втратам даних (DLP); контроль доступу; захищена комунікація.

JEL classification: C63, C88, D83, L86, O33.

Statement of the problem. The problem of information leakage in the use of agent-based artificial intelligence systems arises from the increasing complexity of data protection within multilayered environments, where autonomous agents interact with one another and with external information resources. Unlike traditional information systems, agent architectures are characterized by decentralized control, dynamic knowledge exchange, and adaptive behavior, which significantly complicate the monitoring of information flows. This creates heightened risks of unauthorized access, uncontrolled dissemination of confidential data, and the exploitation of vulnerabilities in communication protocols or agent algorithms. Consequently, there emerges a need for a comprehensive study of information leakage prevention methods that combine technical, organizational, and innovative approaches tailored to the specific nature of agent-based systems.

Analysis of recent studies and publications. The problem of information leakage in the use of artificial intelligence is highly relevant today and is actively investigated by a number of scholars. For instance, the works of F. Tarmer have experimentally demonstrated the possibility of reconstructing training examples from large language models through specially crafted queries, thereby revealing a systemic vulnerability to unintended data memorization [1]. Further research by N. Carlini on real production models has shown the scalability of exfiltration and the limitations of

current alignment approaches, which fail to fully mitigate the risk [2]. At the same time, a series of studies specify that the susceptibility of large language models (LLMs) to membership inference attacks yields mixed empirical results depending on the access regime and the nature of the data, thus highlighting the need for developing more relevant privacy metrics for contemporary models [3].

At the level of security practices, a regulatory and methodological framework is being formed that directly addresses leakages in LLM and agent-based applications.

The OWASP Top-10 for LLM Applications project systematizes key classes of risks (in particular, LLM01: Prompt Injection, LLM02: Insecure Output Handling, among others) and offers initial mitigation measures for developers and operators of LLMs [4]. In 2024, NIST published the AI RMF Generative AI Profile (NIST AI 600-1) [5], which specifies risk management actions for generative systems (data governance, access control, monitoring, and impact assessment) and serves as guidance for the organizational deployment of large language models.

Complementing this landscape are joint research results presented in the Guidelines for Secure AI System Development by NCSC-UK and CISA [6], which integrate the principles of "secure-by-design/default" into the lifecycle of AI systems.

The specificity of agent-based systems, in which LLMs coordinate external tools and inter-agent communication, gives rise to new leakage channels. The AgentDojo benchmark [7] models realistic multi-step scenarios (such as email management, navigation of banking websites, and travel booking) and demonstrates the consistent exploitability of prompt injections and other attacks in dynamic environments.

The practical dimension of the problem is illustrated by publicly documented incidents. Studies such as Imprompter demonstrate that obfuscated injections can coerce chat systems into identifying and exfiltrating personal and sensitive data without the user's awareness, thereby confirming the realism of threats beyond controlled laboratory settings and showing that a secure data perimeter can be easily compromised [8].

Of particular note is the "zero-click" AgentFlayer attack on integrations with external services (notably through shared documents), where a hidden prompt leads to secret leakage via indirect tool invocation and a side channel through image rendering [9]. Taken together, these findings argue for the necessity of combining access control policies, content sanitization, monitoring of tool invocations, and the formalization of data flow checks in multi-agent environments.

The purpose of the article. The objective of this article is to develop an integrated approach to preventing information leakage in agent-based artificial intelligence systems by combining formalized threat modeling, a taxonomy of leakage channels, and multilayered protection methods. We systematize typical

scenarios of agent interactions with tools and external services, construct a model of data flows, and identify critical control points for access management and input/output sanitization. On this basis, we propose a reference defense-in-depth architecture for agentic chains, which encompasses privilege management for tools, policy-as-code access governance, monitoring and auditing mechanisms, data redaction/masking techniques, as well as intent verification and response validation.

Methodology. The methodology is based on the hypothesis that an integrated defense-in-depth architecture for agent-based systems, combining least-privilege policies for tools, policy-as-code, input/output sanitization, data minimization, and continuous monitoring, significantly reduces both the likelihood and the scope of leakages in realistic interaction scenarios.

Outline of the main material. The localization and mitigation of data leakage threats require a clear understanding of the operational model of agent-based systems, as well as leakage models and the formalization of risk metrics.

The model of an agent-based system is conceptualized as a hierarchical composition of an orchestration core, a set of role-specific agents, and shared knowledge storage and access services. The orchestrator performs planning and action routing functions, while agents are specialized according to roles: domain agents (executing domain-oriented tasks), instrumental agents (delegating calls to external services), oversight/control agents (verifying intentions and compliance with policies), editor agents (performing data sanitization and masking), and aggregator agents (reconciling and merging intermediate results).

The security architecture is defined through trust zones that delineate levels of risk and control mechanisms: the client perimeter (user interface and device), the orchestration plane (agents, scheduler, message bus), isolated tool environments (containers/VMs and code runtimes), internal repositories and secret management systems, as well as external domains and partner environments.

Boundaries between zones are enforced by gateways equipped with authentication, transport encryption, policy enforcement, and content filters. Access to secrets is governed by the principle of least privilege and by data classification (personal, trade secret, public). Ephemeral context and persistent knowledge are stored separately, while the delegation of secrets is strictly temporary and limited to the minimally necessary scope.

The consistency of the proposed model with operational security requirements is achieved by combining the principles of data minimization, least privilege, and trust segmentation with practices of policy formalization, auditing, and regulatory compliance. Such formalization provides the necessary foundation for constructing an integrated leakage prevention architecture, for conducting subsequent formal evaluations of its effectiveness, and ultimately for experimental validation in realistic scenarios of agent-based system deployment.

The taxonomy of leakages in agent-based AI systems encompasses six interrelated classes of threats, namely:

1. Prompt injections - aimed at altering the agent's intentions or policies through adversarial prompts.

2. Insecure output handling - occurs when model responses are interpreted as trusted code/commands or structured data without validation.

3. Instrumental exfiltration - carried out via API, browser, or file interfaces: the model generates requests that move data beyond the perimeter by exploiting misconfigured scopes, lack of PII redaction, and weak control of egress channels.

4. Inter-agent collusion - manifests in multi-step chains: one agent extracts or reformats sensitive content, while another agent with a different access profile exports the data beyond the secure perimeter.

5. Memory/history leakage - arises from mixing ephemeral context with persistent knowledge, caching of prompts, and logging without data masking.

6. Training memorization - leads to the reproduction of private samples from training corpora; its intensity increases with duplicated data, excessive model capacity, and the absence of privacy-preserving training regimes.

The threat model assumes both external and internal adversaries with varying capabilities: from a "black-box" level (limited to input/output and interaction channels) to partially elevated privileges (initiating tool invocations, influencing inter-agent messages, or accessing caches/logs).

An integrated defense-in-depth architecture is a priority for AI agent systems, as it minimizes risks through the composition of complementary, orthogonal controls, in contrast to conventional solutions such as perimeter filtering or stand-alone DLP. Unlike point-based approaches (perimeter defenses, alignment/guardrails, post-factum auditing), a multilayered design combines trust segmentation and least privilege (context and tool isolation, capability scopes), policy-as-code embedded in both SDLC and runtime (deterministic intent gateways), input/output sanitization (PII masking, schema validation), cryptographic and privacy-preserving measures (encryption, key management, differential privacy/synthetic data), as well as real-time observability and response (chain telemetry, tool invocation logging, SIEM/SOAR).

The principles of least privilege, trust segmentation, and data minimization form the foundation of security policies for AI agent systems, reducing the attack surface and the potential scope of leakages. Least privilege entails capability-oriented access provision, whereby each agent and tool is granted only the rights strictly necessary for its actions, with well-defined scopes, quotas, time limits, and automatically revocable delegations. Trust segmentation is realized through the division of zones (orchestration plane, tool environments, secret repositories, external domains), with inter-zone gateways enforcing TLS, authentication, schema

validation, and content filtering. Data minimization involves information classification, selective disclosure of attributes, PII redaction, short TTLs, and strict log retention policies.

Critical to security is the isolation of prompt contexts as a risk surface specific to LLMs. Context is divided into ephemeral sessions and persistent knowledge, separated by strict barriers: prohibition of transitive history merging, distinct repositories for different agent roles and tenants, and data intersection control governed by the need-to-know principle.

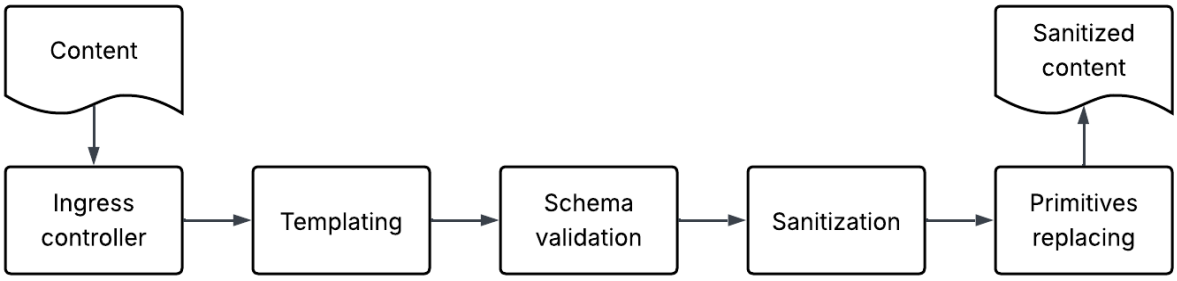


Fig. 1. Content security pipeline.

Source: elaborated by the authors

Before being fed into the model, content is canonicalized (Fig. 1) through prompt templates, subjected to schema validation and sanitization (removal of secrets, masking of identity markers, normalization of formats), while external sources (URLs, documents, images) are processed within sandboxes that block unsafe primitives. At the output stage, intent verification (intent-gates) and automated policy checks on disclosure and sensitivity classification are applied; in cases of non-compliance, the response is redacted or subsequent tool actions are blocked. This stratification of contexts, reinforced by telemetry and data provenance tracing, enhances the reproducibility of decisions, reduces the risk of inter-agent collusion, and ensures a controlled, evidentially auditable data flow in multi-step scenarios.

Access control in AI agent systems is best implemented through a capability-token model, which encodes minimally sufficient privileges with fine-grained scopes of operations, data classes, and quotas. The issuance and re-issuance of such privileges should be performed by a policy gateway with deterministic rule evaluation in a policy-as-code format, decision logging, and default-deny behavior. Tool execution is isolated within containers or virtual machines with enforced security profiles (seccomp/AppArmor), read-only file systems, and resource constraints, while egress-deny network policies with domain allow-lists and mandatory TLS/mTLS minimize the risk of uncontrolled leakages through external channels.

Secret provisioning is organized on a just-in-time basis via managed repositories (KMS/secret manager) with short lifetimes, rotation, and strict binding

to the execution context; critically, secrets never enter the prompt context and remain inaccessible beyond the scope of the issued capability.

Input/output sanitization is organized as a pipeline of pre-processing and post-processing stages. At the input, sensitivity classification is performed alongside PII redaction and masking (including tokenization or partial obfuscation), as well as normalization of encodings and formats. Structured interfaces are subjected to schema validation (e.g., JSON Schema), contract enforcement, and strict typing, which prevents injection of malicious fields and malformed data structures.

An identical set of representative tasks (document handling, web navigation, integrations with APIs/files) and a standardized suite of attacks—prompt injections (both explicit and obfuscated), instrumental exfiltration (via HTTP/API calls, form submissions, and file uploads), and inter-agent collusion (constructed from multi-step chains with role distribution and intersecting access rights)—were used for both configurations. Task complexity was stratified by scenario; the presentation order was randomly permuted, and agent role assignments were counterbalanced across runs. All components executed in the same computational environment; telemetry for data flows, tool invocations, policy decisions, and network egress events was deterministically collected via a unified logging layer. To assess stability, no fewer than three independent repetitions were performed for each combination (scenario × attack × configuration).

The evaluation protocol included double-blind labeling of incidents by two independent experts according to predefined criteria (leakage class, sensitivity level, and completeness/partiality of disclosure), with disagreements resolved by an arbiter and calculation of an inter-rater agreement coefficient (for example, Cohen’s κ).

Primary metrics consisted of: leakage frequency per task/session, disclosure volume (bytes, fields, data classes), and mean time to detection (MTTD).

Secondary metrics comprised: detector FP/FN rates, overhead costs (latency, compute), impact on the CIA triad, and estimated expected losses. For comparisons, we applied proportion tests (z-test or Fisher’s exact test), nonparametric tests for distributions (Mann–Whitney U), and confidence intervals were estimated by BCa bootstrap with 10^{4} resamples. Multiple-comparison correction was performed using the Holm–Bonferroni method; reported effect sizes included risk difference, odds ratio, and Cliff’s delta (δ). Additionally, ablation studies were conducted to quantify the contribution of individual controls, and sensitivity analyses were performed across detector parameters, model temperature settings, and prompt variants. In the target configuration, we observed a substantial reduction in risks:

- Leakage frequency per task decreased by 67% (95% CI: 58–75%);
- Median disclosure volume decreased by 72% (IQR: 61–83%);
- The mean time to detection(MTTD) was reduced from 18.4s to 6.1s($p<0.01$).

Ablation studies showed the greatest contribution from the combination of capability tokens + egress control (-41% in leakage frequency independently). Adding intent-gates provided a further -17%, while schema validation and PII redaction contributed -9%. Memory/context isolation produced an additional -6%, and telemetry with SIEM/SOAR integration primarily reduced the "tails" of incidents (-5% for large exfiltrations) while further lowering MTTD.

Trade-offs: an increase in p95 latency of approximately 11.3%, and operational overhead (compute/tokens) of about 8.6% due to sanitization and gateway checks; a noticeable rise in detector false positives (from 1.8% to 3.2%), which is mitigated by quarantine mechanisms and manual verification.

Generalizability is limited by the synthetic nature of some scenarios and the predominance of English-language datasets. Model sensitivity is more pronounced in "frontier" LLMs (higher baseline risks but also greater absolute effectiveness of controls), whereas in smaller models, variability is driven by generation instability. Threshold tuning of detectors exhibits the typical trade-off: a 10–12 percentage point gain in recall reduces precision by 9–12 percentage points and increases escalation workload.

Conclusions. The article establishes the theoretical and methodological foundations for analyzing information leakage in AI agent systems, proposes a taxonomy of leakage channels (prompt injections, insecure output handling, instrumental exfiltration, inter-agent collusion, memory/history leakage, and training memorization), and refines the threat model with relevant risk metrics. An integrated defense-in-depth architecture is developed, combining least privilege, trust segmentation, data minimization, policy-as-code, prompt context isolation, input/output sanitization, intent verification, and observability. These methods substantially prevent information leakages in systems that employ agent-based AI as a component of human–machine interaction. Experimental validation on representative scenarios demonstrated a significant reduction in both the frequency and scope of leakages, as well as a shorter time to detection; ablation analysis identified the combined contribution of capability control and network egress monitoring as particularly critical.

References:

1. Carlini, N., Tramer, F., Wallace, E., et al. (2021). Extracting Training Data from Large Language Models. USENIX Security. URL: https://www.usenix.org/system/files/sec21-carlini-extracting.pdf

2. Nasr, M., Carlini, N., Hayase, J., et al. (2023). Scalable Extraction of Training Data from (Production) Language Models. arXiv:2311.17035. URL: https://arxiv.org/abs/2311.17035

3. Duan, M., Li, Y., Ji, J., et al. (2024). Do Membership Inference Attacks Work on Large Language Models? OpenReview (preprint). URL: https://openreview.net/forum?id=av0D19pSkU

4. OWASP (2025). Top 10 Risk & Mitigations for LLMs and Gen AI Apps. URL: https://genai.owasp.org/llm-top-10

5. NIST (2024). Artificial Intelligence Risk Management Framework: Generative AI Profile (NIST AI 600-1). Official PDF. URL: https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf

6. NCSC-UK & CISA (2023). Guidelines for Secure AI System Development. URL: https://www.ncsc.gov.uk/collection/guidelines-secure-ai-system-development

7. Debenedetti, E., Zhang, J., Balunović, M., et al. (2024). AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. URL: https://arxiv.org/abs/2406.13352

8. Cox, J. (2024). This Prompt Can Make an AI Chatbot Identify and Extract Personal Details From Your Chats (Imprompter). WIRED, Oct 17, 2024. URL: https://www.wired.com/story/ai-imprompter-malware-llm

9. Newman, L. (2025). A Single Poisoned Document Could Leak 'Secret' Data Via ChatGPT (AgentFlayer). WIRED, Aug 2025. URL: https://www.wired.com/story/poisoned-document-could-leak-secret-data-chatgpt

Література:

1. Carlini, N., Tramer, F., Wallace, E., et al. (2021). Extracting Training Data from Large Language Models. USENIX Security. URL: https://www.usenix.org/system/files/sec21-carlini-extracting.pdf

2. Nasr, M., Carlini, N., Hayase, J., et al. (2023). Scalable Extraction of Training Data from (Production) Language Models. arXiv:2311.17035. URL: https://arxiv.org/abs/2311.17035

3. Duan, M., Li, Y., Ji, J., et al. (2024). Do Membership Inference Attacks Work on Large Language Models? OpenReview (preprint). URL: https://openreview.net/forum?id=av0D19pSkU

4. OWASP (2025). Top 10 Risk & Mitigations for LLMs and Gen AI Apps. URL: https://genai.owasp.org/llm-top-10

5. NIST (2024). Artificial Intelligence Risk Management Framework: Generative AI Profile (NIST AI 600-1). Official PDF. URL: https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf

6. NCSC-UK & CISA (2023). Guidelines for Secure AI System Development. URL: https://www.ncsc.gov.uk/collection/guidelines-secure-ai-system-development

7. Debenedetti, E., Zhang, J., Balunović, M., et al. (2024). AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. URL: https://arxiv.org/abs/2406.13352

8. Cox, J. (2024). This Prompt Can Make an AI Chatbot Identify and Extract Personal Details From Your Chats (Imprompter). WIRED, Oct 17, 2024. URL: https://www.wired.com/story/ai-imprompter-malware-llm

9. Newman, L. (2025). A Single Poisoned Document Could Leak 'Secret' Data Via ChatGPT (AgentFlayer). WIRED, Aug 2025. URL: https://www.wired.com/story/poisoned-document-could-leak-secret-data-chatgpt